

PROGRAMACION

2 - UTN

Proyecto: Food Store

Estudiantes:

- Gonzalo Garrido Fäs

Fecha de entrega: 22/06/2026

Marco Teórico

- *Java 21*

El proyecto fue desarrollado en Java 21, versión LTS (Long Term Support) del lenguaje. Java es un lenguaje orientado a objetos, fuertemente tipado y multiplataforma, que se ejecuta sobre la JVM (Java Virtual Machine). Su uso en este proyecto permite aplicar los pilares de la Programación Orientada a Objetos de manera nativa: encapsulamiento, herencia, polimorfismo y abstracción.

Se utilizaron características modernas del lenguaje como expresiones switch (switch expressions con ->), métodos de instancia con inferencia de tipo, y clases de la API estándar como ArrayList, LocalDate y LocalDateTime.

- *Programación Orientada a Objetos (POO)*

La POO es el paradigma central del proyecto. Sus conceptos aplicados son:

Encapsulamiento: todos los atributos de las entidades son privados y se accede a ellos mediante getters y setters, protegiendo el estado interno de los objetos.

Herencia: se definió una clase abstracta Base con los atributos comunes (id, eliminado, createdAt) de la que heredan todas las entidades (Categoria, Producto, Usuario, Pedido, DetallePedido), evitando duplicación de código.

Polimorfismo: la interfaz Calculable define el método calcularTotal(), que es implementado específicamente por la clase Pedido. Esto permite que distintas clases puedan implementar el mismo comportamiento de manera particular.

Abstracción: la clase Base es abstracta, forzando a que solo pueda ser instanciada a través de sus subclases concretas.

- *Colecciones (Java Collections Framework)*

Dado que el sistema no utiliza base de datos, toda la información se almacena en memoria durante la ejecución del programa mediante el uso del Java Collections Framework. Se utilizó específicamente ArrayList<T>, que permite:

Almacenamiento dinámico de objetos sin tamaño fijo.

Recorrido con ciclos for-each.

Búsqueda, filtrado y eliminación de elementos en tiempo de ejecución.

Cada servicio (CategoriaService, ProductoService, UsuarioService, PedidoService) administra su propia colección en memoria, simulando lo que en un sistema real sería una tabla de base de datos.

- *Interfaces*

Se implementó la interfaz Calculable, que define el contrato del método calcularTotal(). Las interfaces en Java permiten estandarizar comportamientos entre clases que no comparten una jerarquía de herencia directa. En este caso, Pedido implementa Calculable para garantizar que el total del pedido siempre se calcule de forma consistente sumando los subtotales de sus detalles activos.

- *Manejo de Excepciones*

El sistema utiliza excepciones propias (checked exceptions) para gestionar los errores de dominio de forma controlada, evitando cierres inesperados del programa:

EntidadNoEncontradaException: se lanza cuando se busca una entidad por ID y no existe o fue eliminada.

StockInvalidoException: se lanza cuando se intenta crear un producto con precio o stock negativo, o cuando la cantidad solicitada en un detalle supera el stock disponible.

ValidacionException: se lanza ante violaciones de reglas de negocio, como mail duplicado, nombre vacío o pedido sin usuario asociado.

Todas las excepciones son capturadas en la capa de menú mediante bloques try-catch, mostrando mensajes claros al operador sin interrumpir el flujo del programa.

- *Arquitectura por Capas*

El proyecto sigue una arquitectura de tres capas con responsabilidades bien definidas:

CapaPaqueteResponsabilidadEntidadesentities / enumsModelo de dominio, atributos y relacionesServiciosserviceLógica de negocio, validaciones y acceso a coleccionesInterfaz de usuariomenuInteracción con el usuario por consola (Scanner)

Esta separación garantiza que el Main solo ensamble las dependencias y que cada capa sea independiente y reutilizable.

- *Soft Delete* (Baja Lógica)

En lugar de eliminar físicamente los objetos de las colecciones, se implementó el patrón soft delete: al "eliminar" una entidad, el campo eliminado heredado de Base se marca como true. Los listados filtran siempre los registros con eliminado = false, manteniendo el historial completo en memoria.

Decisiones Técnicas y Arquitectura:

Abordaje de la Solución

El desarrollo del sistema de gestión de pedidos **Food Store** se diseñó bajo el paradigma de la Programación Orientada a Objetos (POO), utilizando **Java 21** como lenguaje base. Debido a que el alcance actual de la materia requiere una persistencia temporal, toda la información del sistema se administra directamente **en memoria** a través del framework de **Colecciones de Java** (List, Map, etc.) , simulando el comportamiento de un repositorio de datos.

La interacción se centralizó en una interfaz dirigida por consola mediante un menú de opciones dinámico alimentado por la clase Scanner. Al no implementarse un login inicial, se adoptó el rol de un usuario operador general capaz de navegar por los submenús de las diferentes épicas del backlog (Categorías, Productos, Usuarios y Pedidos).

Organización de Paquetes (Arquitectura por Capas)

Para garantizar una clara separación de responsabilidades y la modularización recomendada por la cátedra, el proyecto se estructuró bajo el patrón de diseño arquitectónico en capas (Entidades + Servicios + UI), distribuyéndose en los siguientes paquetes bajo la raíz src/integrado/prog2/:

- **entities (Modelo de Dominio):** Contiene las clases del diagrama UML. Se implementó una clase abstracta común llamada Base (con atributos compartidos como id, eliminado y createdAt) de la cual heredan Categoría, Producto, Usuario, Pedido y DetallePedido. También se incluyó la sobrescritura de métodos fundamentales como toString() para dar un formato prolijo a los listados por pantalla.
- **enums (Enumeraciones):** Define los tipos de datos enumerados solicitados por el modelo de negocio para estructurar las propiedades estáticas: Rol (ADMIN, USUARIO), Estado (PENDIENTE, CONFIRMADO, etc.) y FormaPago (TARJETA, EFECTIVO, etc.).
- **interfaces (o dentro de entities):** Incluye la interfaz Calculable y su método abstracto calcularTotal(), implementado de manera obligatoria en la clase Pedido para calcular el coste final de las compras en base a sus líneas de detalle.
- **services (Capa de Lógica de Negocio):** Clases encargadas de coordinar los casos de uso, aplicar las reglas mínimas del negocio (p. ej., verificar la unicidad del email de un usuario o que el stock no sea negativo) y manipular las colecciones en memoria.
- **exception (Manejo de Errores):** Paquete con las excepciones personalizadas construidas para robustecer la aplicación (ej: EntidadNoEncontradaException, StockInvalidoException).
- **Main.java (Capa de Presentación / UI):** Punto de entrada del sistema que gestiona el flujo de los menús visuales por consola, captura los ingresos de datos por teclado y atrapa los errores para evitar cierres abruptos de la aplicación.

Fotos del proyecto:

- Main Principal:

```
Output - FoodStore (run)

run:

=====
=== FOOD STORE  SISTEMA DE PEDIDOS ===
=====

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir

-----
Seleccione:
```

- Menu de categoría y creación de producto:

```
=====
GESTI N DE CATEGOR AS
=====

1. Listar categor as
2. Crear categor a
3. Editar categor a
4. Eliminar categor a
0. Volver al men principal

-----
Seleccione: 2

--- CREAR CATEGOR A ---
Nombre: Mango
Descripción: Fruta
[OK] Categor a creada con ID: 1

Presione Enter para continuar...
```

- **Menu y creación de producto:**

```
=====
=== FOOD STORE 🎲 SISTEMA DE PEDIDOS ===
=====

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir

-----

Seleccione: 2

=====

GESTIÓN DE PRODUCTOS
=====

1. Listar productos
2. Listar por categoría
3. Crear producto
4. Editar producto
5. Eliminar producto
0. Volver al menú principal

-----

Seleccione: 3

--- CREAR PRODUCTO ---
Categorías disponibles:
  ID: 1   | Nombre: Mango   | Descripción: Fruta
ID de categoría: 1
Nombre: Durazno
Descripción: Fruta
Precio: 500
Stock: 5
Imagen (URL o nombre, Enter para omitir):
🎲 Disponible? (S/N): s
[OK] Producto creado con ID: 2

Presione Enter para continuar...|
```

- **Menu usuario y creación de usuario:**

```
=====
          GESTION DE USUARIOS
=====

1. Listar usuarios
2. Crear usuario
3. Editar usuario
4. Eliminar usuario
0. Volver al menú principal
-----

Seleccione: 2

--- CREAR USUARIO ---
Nombre: Marcos
Apellido: Perez
Mail: gononossog@gmail.com
Celular: 351-----
Contraseña: MarcosPerezElCapo10
Roles: 1=ADMIN 2=USUARIO
Seleccione rol: 2
[OK] Usuario creado con ID: 3

Presione Enter para continuar...|
```


- Menu pedido y solicitud:

```
=====
=== FOOD STORE ♦ SISTEMA DE PEDIDOS ===
=====

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir

-----
Seleccione: 4

=====
GESTIÓN DE PEDIDOS
=====

1. Listar pedidos
2. Listar pedidos por usuario
3. Crear pedido con detalles
4. Actualizar estado / forma de pago
5. Eliminar pedido
0. Volver al menú principal

-----
Seleccione: 3

--- CREAR PEDIDO ---
Usuarios disponibles:
ID: 3 | Nombre: Marcos | Apellido: Perez | Mail: gononossog@gmail.com | Celular: 351----- | Rol: USUARIO
ID del usuario: 3
Formas de pago: 1=TARJETA 2=TRANSFERENCIA 3=EFFECTIVO
Seleccione: 1

Productos disponibles:
ID: 2 | Nombre: Durazno | Precio: $500.00 | Stock: 5 | Disponible: Si | Categoría: Mango
ID del producto: 2
Cantidad: 4
♦Agregar otro producto? (S/N): n
[OK] Pedido creado con ID: 4 | Total: $2000.00

Presione Enter para continuar...|
```

- Menu de salida:

```
=====
=== FOOD STORE ♦ SISTEMA DE PEDIDOS ===
=====

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir

-----
Seleccione: 0

♦Hasta luego! Food Store cerrado.
```

Complicaciones

Durante el transcurso del desarrollo técnico del TPI surgieron ciertos desafíos lógicos debido a las restricciones de persistencia en memoria y las reglas de negocio planteadas. A continuación, se detallan las principales problemáticas y los mecanismos implementados para resolverlas:

1. **Dificultad:** Consistencia de datos en la creación fallida de Pedidos.

Obstáculo: Al procesar la creación de un pedido con múltiples ítems (HU-PED-02), si una línea de detalle fallaba en medio del proceso (por ejemplo, por ingresar un producto con stock insuficiente), los detalles previos ya procesados se quedaban flotando de forma inconsistente en memoria.

Solución: Se implementó un manejo defensivo mediante bloques try-catch dentro del servicio de pedidos. Si el método `addDetallePedido(...)` arroja una excepción propia de falta de stock, el error es capturado inmediatamente, se revierte la operación cancelando la construcción del pedido en memoria y se limpia la instancia temporal para salvaguardar la integridad del inventario de productos.

2. **Dificultad:** Control de Integridad referencial con la Baja Lógica (Soft Delete).

Obstáculo: Al marcar una entidad (como un Usuario o una Categoría) con `eliminado = true` para cumplir con la política de no remoción física, existía el riesgo de permitir que un operador asociara nuevos productos o pedidos a estas entidades que teóricamente ya "no existían".

Solución: Se modificaron los métodos de búsqueda (`findById`) y los listados dentro de la capa de servicios para que siempre validen la compuerta lógica del atributo binario `eliminado`. Si el elemento posee la bandera en `true`, el servicio lanza de forma automatizada una excepción del tipo `EntidadNoEncontradaException`, impidiendo que el menú exponga o reutilice registros dados de baja.

3. **Dificultad:** Validación rigurosa de entradas de teclado en la UI.

Obstáculo: El uso repetitivo de `Scanner` para valores numéricos (`nextInt()`, `nextDouble()`) dejaba saltos de línea pendientes en el búfer de entrada, lo que provocaba que el menú saltara campos de texto (`nextLine()`) en las pantallas consecutivas de creación de datos, además de producirse fallos críticos del tipo `InputMismatchException` si el usuario digitaba letras en campos de ID.

Solución: Se diseñó una clase de utilidad para lectura de datos en el paquete de UI, la cual lee siempre las líneas completas como cadenas de texto (`nextLine()`) y realiza la conversión manual a tipos numéricos mediante bloques de control de excepciones

(Integer.parseInt()). Si la conversión falla, limpia el canal informando un mensaje amigable y solicita el reingreso del dato sin romper el ciclo del menú principal.